

Desenvolvimento de Aplicação Web Front-End

Curso: Ciência da Computação

Flávia Martins

Dom Helder

O que é o DOM?

DOM significa **Document Object Model**.

Ele representa o documento HTML como uma estrutura de objetos em forma de árvore. Cada tag HTML pode ser acessada e manipulada pelo JavaScript.

Na prática

Quando usamos `document.getElementById("lista-cursos")`, estamos pedindo ao navegador para encontrar um elemento dentro dessa árvore.

HTML vira uma árvore

Exemplo conceitual:

- document
 - html
 - head
 - body
 - header
 - main
 - section
 - article
 - button

Ou seja...

Selecionar elementos é navegar nessa árvore.

Primeiras alterações no DOM

```
// Alterando o fundo da página
document.body.style.backgroundColor = "#f4f6f8";

// Alterando o primeiro h1 encontrado
document.getElementsByTagName("h1")[0].textContent =
  "Aprenda na prática";
```

Selecionar antes de manipular

O fluxo mais comum em JavaScript para páginas web é:

- 1 selecionar um elemento;
- 2 guardar esse elemento em uma variável;
- 3 modificar texto, estilo, classe, atributo ou comportamento.

Modelo mental

Selecionar → Guardar → Modificar

Principais métodos de seleção

Método	Retorna	Uso comum
<code>getElementById()</code>	Elemento único	ID
<code>getElementsByTagName()</code>	HTMLCollection	Tag
<code>getElementsByClassName()</code>	HTMLCollection	Classe
<code>querySelector()</code>	Primeiro elemento	Seletor CSS
<code>querySelectorAll()</code>	NodeList	Seletor CSS

Dica

Para projetos modernos, `querySelector` e `querySelectorAll` costumam ser os mais flexíveis.

Selecionando por ID

O método `getElementById()` seleciona um único elemento pelo atributo `id`.

Quando usar?

Use quando o elemento tem uma identificação única na página.

Exemplo: alterar o título da plataforma

```
const titulo = document.getElementById("titulo-plataforma");  
  
console.log(titulo.textContent);  
  
titulo.textContent = " Plataforma Interativa";
```


Exemplo: alterar o placeholder da busca

```
const campoBusca = document.getElementById("busca-aula");  
campoBusca.placeholder = "Busque por curso, modulo ou aula...";
```

Exemplo: atualizar progresso

```
const aulasConcluidas = document.getElementById("aulas-concluidas");  
const barraProgresso = document.getElementById("barra-progresso");  
  
aulasConcluidas.textContent = "3";  
barraProgresso.style.width = "30%";
```

Selecionando por nome de tag

O método `getElementsByName()` seleciona todos os elementos com uma determinada tag.

Exemplo

Selecionar todos os `h2` dos cards para padronizar títulos de cursos.

Ele retorna uma **HTMLCollection** - serve para armazenar e permitir o acesso a vários elementos HTML ao mesmo tempo.

Exemplo: acessar títulos dos cards

```
const titulos = document.getElementsByTagName("h2");  
  
console.log(titulos[0].textContent);  
console.log(titulos[1].textContent);  
console.log(titulos[2].textContent);
```

Exemplo: alterar todos os h2

```
const titulos = document.getElementsByTagName("h2");

for (let i = 0; i < titulos.length; i++) {
  titulos[i].style.fontWeight = "700";
  titulos[i].style.color = "#1f2937";
}
```

Selecionando por classe

O método `getElementsByClassName()` seleciona todos os elementos que possuem uma classe CSS específica.

Na plataforma

Podemos selecionar todos os cards com a classe `card-curso` ou todos os botões com a classe `btn`.

Ele também retorna uma **HTMLCollection**.

Exemplo: selecionar cards

```
const cards = document.getElementsByClassName("card-curso");  
  
console.log(cards.length);  
console.log(cards[0].textContent);
```

Exemplo: destacar o primeiro card

```
const cards = document.getElementsByClassName("card-curso");  
  
cards[0].style.backgroundColor = "#e0f2fe";  
cards[0].style.border = "2px solid #0284c7";
```


Exemplo: alterar todos os botões

```
const botoes = document.getElementsByClassName("btn");

for (let i = 0; i < botoes.length; i++) {
  botoes[i].textContent = "Come ar agora";
  botoes[i].style.cursor = "pointer";
}
```

Por que usar querySelector?

O método `querySelector()` aceita a mesma lógica dos seletores CSS.

- `#id` seleciona por ID;
- `.classe` seleciona por classe;
- `tag` seleciona por tag;
- `[atributo]` seleciona por atributo;
- `pai filho` seleciona descendentes;
- `:first-child` e outras pseudo-classes também podem ser usadas.

Importante

`querySelector()` retorna apenas o primeiro elemento encontrado.

Exemplo: primeiro card em destaque

```
const cardDestaque = document.querySelector(".card-curso.destaque");  
  
cardDestaque.style.transform = "scale(1.02)";  
cardDestaque.style.boxShadow = "0 10px 20px rgba(0, 0, 0, 0.12)";
```

Exemplo: botão dentro do card em destaque

```
const botaoDestaque = document.querySelector(  
  ".card-curso.destaque .btn-iniciar"  
);  
  
botaoDestaque.textContent = "Continuar aula";
```

Exemplo: selecionar por atributo

```
const cursoFrontend = document.querySelector(  
  '[data-categoria="frontend"]'  
);  
  
cursoFrontend.style.borderColor = "#2563eb";
```

querySelectorAll

O método `querySelectorAll()` também usa seletores CSS, mas retorna todos os elementos correspondentes.

Retorno

Ele retorna uma **NodeList**, que pode ser percorrida com `forEach()`.

Exemplo: selecionar todos os cards

```
const cards = document.querySelectorAll(".card-curso");

cards.forEach((card) => {
  console.log(card.textContent);
});
```

Exemplo: estilizar todos os cards

```
const cards = document.querySelectorAll(".card-curso");

cards.forEach((card) => {
  card.style.borderRadius = "16px";
  card.style.padding = "24px";
  card.style.transition = "0.2s ease";
});
```


Exemplo: selecionar botões dentro dos cards

```
const botoesIniciar = document.querySelectorAll(
  ".card-curso .btn-iniciar"
);

botoesIniciar.forEach((botao) => {
  botao.textContent = "Acessar conte do";
});
```

Elemento único ou coleção?

Alguns métodos retornam um único elemento. Outros retornam uma coleção.

Retorno	Exemplo de método
Elemento único	<code>getElementById()</code>
Primeiro elemento encontrado	<code>querySelector()</code>
HTMLCollection	<code>getElementsByClassName()</code>
HTMLCollection	<code>getElementsByTagName()</code>
NodeList	<code>querySelectorAll()</code>

Cuidado

Uma coleção não é um elemento. Para manipular seus itens, é preciso acessar índices ou percorrer a coleção.

Erro comum: tentar alterar uma coleção

```
const cards = document.getElementsByClassName("card-curso");  
  
// Erro conceitual:  
// cards.style.backgroundColor = "yellow";  
  
// Correto: acessar um item específico  
cards[0].style.backgroundColor = "yellow";
```

Percorrendo HTMLCollection com for

```
const cards = document.getElementsByClassName("card-curso");

for (let i = 0; i < cards.length; i++) {
  cards[i].classList.add("card-ativo");
}
```

Percorrendo NodeList com forEach

```
const cards = document.querySelectorAll(".card-curso");

cards.forEach((card) => {
  card.classList.add("card-ativo");
});
```

Coleções “ao vivo” e coleções estáticas

- `getElementsByClassName()` e `getElementsByTagName()` retornam **HTMLCollection**.
- Em geral, **HTMLCollection** é uma coleção ao vivo.
- `querySelectorAll()` retorna **NodeList**.
- A **NodeList** de `querySelectorAll()` é estática.

Tradução prática

Se novos elementos forem adicionados ao DOM, uma **HTMLCollection** pode refletir essa mudança automaticamente. Uma **NodeList** criada antes não será atualizada automaticamente. Ao vivo: atualizada automaticamente pelo navegador sempre que o DOM é modificado.

Exemplo: HTMLCollection ao vivo

```
const cards = document.getElementsByClassName("card-curso");

console.log(cards.length);

const novoCard = document.createElement("article");
novoCard.className = "card-curso";
novoCard.textContent = "Novo curso";

document.getElementById("lista-cursos").appendChild(novoCard);

console.log(cards.length); // Atualizado automaticamente
```

Exemplo: NodeList estática

```
const cards = document.querySelectorAll(".card-curso");

console.log(cards.length);

const novoCard = document.createElement("article");
novoCard.className = "card-curso";
novoCard.textContent = "Novo curso";

document.getElementById("lista-cursos").appendChild(novoCard);

console.log(cards.length); // Mantém o valor original
```


O que podemos manipular?

Depois de selecionar um elemento, podemos alterar:

- texto;
- HTML interno;
- classes CSS;
- estilos inline;
- atributos;
- propriedades de formulário;
- eventos.

Regra prática

Prefira mudar classes CSS com `classList` em vez de aplicar muitos estilos diretamente com `style`.

textContent: alterar texto

```
const subtítulo = document.querySelector(".subtítulo");  
subtítulo.textContent =  
  "Organize seus estudos e acompanhe seu progresso.";
```

innerHTML: inserir HTML

```
const painel = document.querySelector(".painel-progresso");

painel.innerHTML += `
  <p class="mensagem-progresso">
    Continue estudando para liberar novos módulos.
  </p>
`;
```

Cuidado com innerHTML

`innerHTML` é útil, mas deve ser usado com atenção.

Risco

Se o conteúdo vier diretamente do usuário, pode abrir espaço para inserção de código malicioso.

Boa prática

Para textos simples, prefira `textContent`. Para criar estruturas, considere `createElement()`.

classList: adicionar e remover classes

```
const cardDestaque = document.querySelector(".card-curso");  
  
cardDestaque.classList.add("selecionado");  
cardDestaque.classList.remove("destaque");  
cardDestaque.classList.toggle("favorito");
```

value: trabalhando com campos de formulário

```
const campoBusca = document.getElementById("busca-aula");  
  
campoBusca.value = "JavaScript";  
  
console.log(campoBusca.value);
```

Boas práticas ao usar seletores

- Use IDs para elementos únicos.
- Use classes para grupos de elementos.
- Use `data-*` para informações específicas do JavaScript.
- Prefira `textContent` para alterar textos.
- Evite repetir seletores muitas vezes: guarde em variáveis.
- Prefira `classList` para mudanças visuais.
- Dê nomes claros às variáveis.

Boa prática: guardar seletores

```
// Menos recomendado
document.querySelector(".subtitulo").textContent = "Novo texto";
document.querySelector(".subtitulo").style.color = "#2563eb";

// Mais organizado
const subtitulo = document.querySelector(".subtitulo");

subtitulo.textContent = "Novo texto";
subtitulo.classList.add("texto-destaque");
```


Boa prática: usar classes CSS

```
// Evite acumular muitos estilos no JavaScript
card.style.backgroundColor = "#e0f2fe";
card.style.border = "2px solid #0284c7";
card.style.borderRadius = "16px";

// Prefira alternar classes
card.classList.add("card-destaque");
```

Boa prática: separar responsabilidades

```
function destacarCard(card) {  
  card.classList.add("card-destaque");  
}  
  
function marcarBotaoComoConcluido(botao) {  
  botao.textContent = "Conclu da";  
  botao.classList.add("concluido");  
}  
  
function obterTituloDoCard(card) {  
  return card.querySelector("h2").textContent;  
}
```

Erros comuns

- esquecer o ponto ao selecionar classe: usar `"card"` em vez de `".card"`;
- esquecer o jogo da velha ao selecionar ID: usar `"titulo"` em vez de `"#titulo"`;
- tentar aplicar `style` diretamente em uma coleção;
- usar `innerHTML` quando `textContent` seria suficiente;
- repetir muito código em vez de criar funções;
- manipular elementos antes do HTML ser carregado.

Quando o script carrega antes do HTML

```
// Op    o 1: colocar o script no fim do body
<script src="script.js"></script>

// Op    o 2: usar defer no head
<script src="script.js" defer></script>
```

Atividade prática: objetivo geral

A atividade será realizada sobre a Plataforma de Estudos já iniciada nas aulas anteriores.

Objetivo

Aplicar seletores JavaScript para localizar, alterar e tornar interativos os elementos principais da plataforma.

Resultado esperado

Ao final, a plataforma deverá permitir destaque de curso, alteração de textos, atualização visual dos botões e acompanhamento básico de progresso.

Organização da prática

A prática será dividida em cinco etapas:

- 1 preparar o HTML com IDs, classes e atributos;
- 2 selecionar elementos principais com JavaScript;
- 3 alterar textos, estilos e classes;
- 4 implementar busca e progresso.

Etapa 1: garantir elementos no HTML

```
<input
  id="busca-aula"
  class="campo-busca"
  type="text"
  placeholder="Buscar aula..."
>

<select id="filtro-categoria">
  <option value="todos">Todos</option>
  <option value="frontend">Front-end</option>
  <option value="algoritmos">Algoritmos</option>
  <option value="banco-dados">Banco de Dados</option>
</select>
```

Etapa 1: revisar os cards

```
<article class="card-curso" data-categoria="frontend">
  <h2>JavaScript para Web</h2>
  <p>DOM, eventos e seletores.</p>
  <button class="btn btn-iniciar">Iniciar aula</button>
</article>
```

Atenção

A classe card-curso permite selecionar todos os cards. O atributo data-categoria permite filtrar os cursos.

Etapa 2: criar o arquivo JavaScript

```
// script.js

const titulo = document.getElementById("titulo-plataforma");
const campoBusca = document.getElementById("busca-aula");
const filtroCategoria = document.getElementById("filtro-categoria");

const cards = document.querySelectorAll(".card-curso");
const botoes = document.querySelectorAll(".btn-iniciar");
```

Etapa 3: alterar informações iniciais

```
titulo.textContent = "CodeStudy - Plataforma de Estudos";

campoBusca.placeholder =
  "Digite o nome do curso ou conte do...";

cards[0].classList.add("destaque");
botoes[0].textContent = "Come ar agora";
```

Etapa 4: progresso da plataforma

```
let aulasConcluidas = 0;
const totalAulas = botoes.length;

const contador = document.getElementById("aulas-concluidas");
const barra = document.getElementById("barra-progresso");

function atualizarProgresso() {
  const percentual = (aulasConcluidas / totalAulas) * 100;

  contador.textContent = aulasConcluidas;
  barra.style.width = `${percentual}%`;
}
```

Desafio 1: ajustes iniciais da plataforma

Usando seletores JavaScript:

- 1 altere o título principal da plataforma;
- 2 altere o placeholder do campo de busca;
- 3 altere o texto do primeiro botão para “Começar agora”;
- 4 destaque visualmente o primeiro card;
- 5 adicione uma classe CSS em todos os cards.

Modelo de solução: Desafio 1

```
const titulo = document.getElementById("titulo-plataforma");
const busca = document.getElementById("busca-aula");
const primeiroBotao = document.querySelector(".btn-iniciar");
const primeiroCard = document.querySelector(".card-curso");
const cards = document.querySelectorAll(".card-curso");

titulo.textContent = "Minha Plataforma de Estudos";
busca.placeholder = "Pesquisar conte do...";
primeiroBotao.textContent = "Come ar agora";
primeiroCard.classList.add("destaque");

cards.forEach((card) => {
  card.classList.add("card-interativo");
});
```

Checklist da atividade prática

A entrega da Plataforma de Estudos deve conter:

- cards selecionados via JavaScript;
- atualização de texto ou classe após interação;
- contador de aulas concluídas;
- barra de progresso atualizada;
- código organizado e legível.

Critérios de avaliação

Critério	O que será observado
Seletores	Uso adequado de IDs, classes, tags e seletores CSS.
Manipulação	Alteração correta de texto, classes, estilos e atributos.
Organização	Código claro, com variáveis bem nomeadas.
Integração	Funcionalidades aplicadas à Plataforma de Estudos.

Resumo

Nesta aula, vimos que:

- o DOM representa a página HTML como uma árvore de objetos;
- seletores permitem encontrar elementos nessa árvore;
- `getElementById()` retorna um elemento único;
- `getElementsByClassName()` e `getElementsByTagName()` retornam coleções;
- `querySelector()` e `querySelectorAll()` usam seletores CSS;